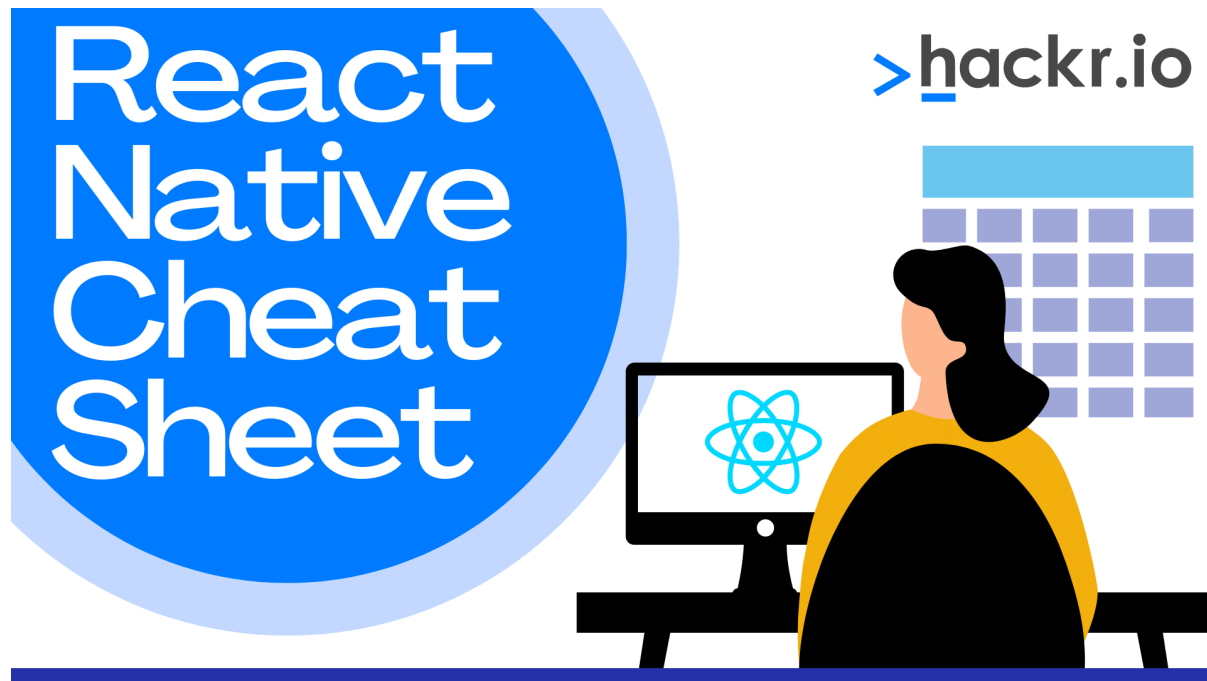




Before we proceed to the nitty-gritty of our cheat sheet, let's quickly brush upon the prerequisites to use React Native.

Prerequisites

Here's what you need to use React Native on your system:

1. Install **Visual Studio Code** (or your code editor of choice).
2. Install **Android Studio**. By default, Android Studio will likely be the most recent Android SDK. You'll need Android 6.0 (Marshmallow) SDK or above for React Native. We recommend you use the most recent SDK.
3. Create routes for the **Java SDK** and **Android SDK** environment variables:
 - a. Enter "Edit the system environment variables" in the Windows search menu to launch the System Properties window.
 - b. Under User variables, choose Environment Variables... and then New...
 - c. Fill in the variable path's name and value. The following are the default directories for the Java and Android SDKs.

ANDROID HOME: C:UsersusernameAppDataLocalAndroidSdk JAVA HOME:
C:Program FilesAndroidAndroid Studiojrebin ANDROID HOME:
C:UsersusernameAppDataLocalAndroidSdk

React Native Command Line

Looking to learn different commands to create a project, run Android or iOS apps, link libraries, and upgrade React Native and NPM packages?

You can with all these React Native commands. Let's dive in.

Creating a Project

To start a new project, open the Terminal App and type these commands:

```
react-native init AppName;  
cd AppName;
```

Running App in the iOS Simulator

Use the following commands in the terminal to run your program on the iOS simulator:

```
react-native run-ios;
```

Running App in the Android Emulator

Use these commands to run your program on the Android emulator:

```
cd AppName;  
react-native run-android;
```

Specifying a Simulator Device

This command will help you start the simulator on an iPhone 7 Plus:

```
react-native run-ios --simulator "iPhone 7 Plus"
```

To list available devices, execute:

```
xcrun simctl list devices
```

Linking Dependencies

Sometimes, you'll need to connect third-party library dependencies. Suppose you have the `react-native-vector-icons` library installed. To add icons fonts to your project, you'd do the following:

```
react-native link
```

Linking Libraries

If you're using libraries like `Linking` or `PushNotifications` on iOS, you'll have to include Header Search Paths. You can also refer to React Native documentation.

Here is how to add a path to the libraries that come with React Native:

1. In the iOS folder, open **AppName.xcodeproj**.
2. Click **AppName** in the left panel.
3. In the **TARGETS** section, click **AppName**.
4. Click **Build Settings**.
5. Within the Search Paths sections, look for the **Header Search Path** row.
6. Look for the term "header." Double-click on a **route** in the AppName column in the Path row inside Search.
7. Click the + button in the popped-up window's left bottom corner.
8. Paste `$(SRCROOT)/../node_modules/react-native/Libraries` into the first column and choose recursive from a dropdown menu in the second column.

Upgrading React Native

To update React Native to the most recent version, run this command:

```
npm install --save react-native@latest;
```

To upgrade project files, execute:

```
react-native upgrade;
```

Upgrading NPM Packages

To list all outdated packages, execute:

```
npm outdated
```

To update a package to the latest version, execute:

```
npm install --save package-name@latest;
```

Development Flow

Enabling Hot Reloading

Enabling hot reloading updates your app in the simulator whenever you make a change to the code.

To enable hot reloading, run your app in the simulator. Once it's up, select **Enable Hot Reloading**. Press ⌘D if you're using the iOS simulator, or ⌘M if you're using the Android emulator.

Enabling Remote Debugging

When remote debugging is enabled, a new tab in Chrome will appear with instructions on how to launch Developer Tools. This will display debug information like problems and warnings. You may also use `console.log('Debug', object)` in your app's code to output custom data, such as strings, variables, objects, components, and so on.

To enable remote debugging, you must first run your app in the simulator. Once it's up, choose Debug JS Remotely by pressing ⌘D if you're using the iOS simulator or ⌘M if you're using the Android emulator.

Code Snippets

Let's explore React Native layout commands with these code snippets.

Detecting Screen Size

You may need to know the screen size of a device that your app is operating on from time to time in order to alter the layout, text sizes, and other aspects of your app.

```
import { Dimensions } from 'react-native';
```

```
const { width, height } = Dimensions.get('window');
```

```
alert(Screen size: ${width}x${height});
```

Here are various screen sizes for your reference:

- **iPhone 5:** 320 x 568
- **iPhone 6:** 375 x 667
- **iPhone 6 Plus:** 414 x 736

Transforming Text

Use the `toUpperCase()` function of string variables to convert text to upper case:

```
<Text>{this.props.title.toUpperCase()}</Text>
```

Use the `toLowerCase()` method of string variables to transform text to lower case:

```
<Text>{this.props.title.toLowerCase()}</Text>
```

Limiting Number of Lines for Text Component

The `numberOfLines` prop helps you limit the number of lines shown for the Text component and cut the text that doesn't fit:

```
<Text numberOfLines={2}>
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam tincidunt eu neque non varius. Phasellus id arcu in enim sodales sodales. Vestibulum maximus hendrerit feugiat. Maecenas nec risus metus. Nam ullamcorper et mauris sit amet placerat. Cras elementum nunc a lorem auctor accumsan. Nam dapibus tristique tempor. Aenean nec augue viverra, tincidunt magna ac, varius leo. Nunc condimentum tristique placerat. Donec ullamcorper nibh vel lacinia pharetra. Praesent a scelerisque tellus. Fusce aliquet porta scelerisque. Suspendisse convallis urna eu arcu finibus ullamcorper. Etiam commodo faucibus ligula, ac consectetur mauris gravida id. Quisque lacus tortor, pellentesque sed eleifend nec, pellentesque vitae diam.

```
</Text>
```

Components

Now, we'll cover how to use various React Native components to develop applications:

Component Boilerplate

You can use this boilerplate to create new components:

```
import React, { Component } from 'react';
import {
  StyleSheet,
  Text,
  View
} from 'react-native';

export default class Component extends Component {

  render() {
```

```

    return (
      <View style={styles.container}>
        <Text>Component</Text>
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
  },
});

```

Default Prop Values

Use the `defaultProps` property to set default prop values for your components.

By supplying a custom value to the component JSX expression, you can always override the default value.

For example, if you use `<header title="My Header" />`, the title value will be changed from default to My Header:

```

import React, { Component } from 'react';
import {
  StyleSheet,
  Text,
  View
} from 'react-native';

export default class Header extends Component {

  static defaultProps = {
    title: 'Default Header'
  }

  render() {
    return (
      <View >
        <Text style={styles.header}>
          {this.props.title.toUpperCase()}

```

```

        </Text>
      </View>
    );
  }

}

const styles = StyleSheet.create({
  header: {
    fontFamily: 'Avenir',
    fontSize: 40,
    fontWeight: 'bold',
    textAlign: 'center',
  },
});

```

Spread Attributes

You can send props as objects in JSX using the spread operator:

```

render() {
  const props = {
    title: 'My Title',
    size: 20,
  };
  return <Header {...props} />;
}

```

This code is essentially the same as follows:

```

render() {
  return <Header title="My Title" size={20} />;
}

```

Pulling Properties out of Objects

You can pull properties out of objects using the `const { prop1, prop1, ...rest } = object` notation:

```

render() {
  const { title, size, ...rest } = this.props;
  return (

```

```

    <View>
      <Text style={{styles.header, { fontSize: size }}} {...rest}>
        {title.toUpperCase()}
      </Text>
    </View>
  );
}

```

Using ES7 Decorators

To use ES7 decorators, you should first install babel-plugin-transform-decorators-legacy first:

```
npm install --save-dev babel-plugin-transform-decorators-legacy
```

Then, edit the .babelrc file to include transform-decorators-legacy plugin:

```

{
  "presets": [
    "react-native"
  ],
  "plugins": [
    "transform-decorators-legacy"
  ]
}

```

We can use the @connect decorator from the react-redux package in the following example:

```

import { connect } from 'react-redux';
import { bindActionCreators } from 'redux';

@connect(
  (state) => {
    const { user, data } = state;
    return {
      user,
      data
    };
  },
  (dispatch) => ({
    actions: { ...bindActionCreators({ actionA, actionB }, dispatch) }
  })
)
export default class App extends Component {

  render({ user } = this.props) {

```



```

return (
  <View >
    <Text style={styles.header}>
      Hello {user.name}
    </Text>
  </View>
);
}
}

```

React Native Stylesheet Cheat Sheet

Ready to style your components? Keep reading!

Applying Multiple Styles to a Component

You can use an array to pass multiple styles to a component:

```
<View style={[styles.generic, styles.specific, { color: 'blue' }]} />
```

Inline Styling

Put the style within the HTML element, as the name says. Let's say we want to put a border around our residence. The color variable determines the border color:

```

function House() {

  const [color, setColor] = useState("red");

  return (

    <div>

      <h2 style={{ "border": `1px solid ${color}` }}>This is a
{color} house</h2>

    </div>

  );

}

```

Alternatively, you can create a style object and pass it in the style attribute like so:

```
function House() {
```

```

    const [color, setColor] = useState("red");

    //style obj contains the css

    const style={

        "border":`1px solid ${color}`

    }

    return (

        <div>

            <h2 style={style}>This is a {color} house</h2>

        </div>

    );
}

```

Making a Circle

The trick is to use half of your circle's width/height for borderRadius:

```

<View style={{
  width: 90,
  height: 90,
  borderRadius: 90 / 2,
  backgroundColor: '#2196F3'
}}/>

```

Fitting Image Into Flexible Container

Let's say you have a flexible container and want to fit a centered picture inside it that fills the entire container:

Here's how it's done:

```

<View style={{

```

```

    backgroundColor: '#dddddd',
    flex: 1,
  }}>
  <Image source={{ uri: 'https://placekitten.com/g/800/600' }} style={{
    position: 'absolute',
    top: 0,
    left: 0,
    bottom: 0,
    right: 0,
  }} />
</View>

```

Flexbox

The primary purpose of Flexbox is to layout the component's children. Now, let's chat more about layout.

Primary and Secondary Axis

The primary axis is always in opposition to the secondary axis. If the primary axis is a column, the secondary will be a row. The vice-versa is also true.

Justify Content

The distribution of children along the principal axis is determined by the content of a component's style. You also have several alternatives available:

1. flex-start (default) — Children distributed at the start.

```

container: {
  flexDirection: 'row',
  justifyContent: 'flex-start'
}

```

2. center — Children are centered.

```

container: {
  flexDirection: 'row',
  justifyContent: 'center'
}

```

3. flex-end — Children distributed at the end.

```
container: {  
  flexDirection: 'row',  
  justifyContent: 'flex-end'  
},
```

4. space-around — Children distributed with equal space around them.



```
container: {  
  flexDirection: 'row',  
  justifyContent: 'space-around'  
}
```

5. space-between — Children distributed evenly.



```
container: {  
  flexDirection: 'row',  
  justifyContent: 'space-between'  
}
```

Align Items

This command sets the alignment of children along the secondary axis. Here's what you can do:

1. flex-start — Children aligned at the start.



```
container: {  
  flexDirection: 'row',  
  justifyContent: 'space-between',  
  alignItems: 'flex-start'  
}
```

2. center — Children aligned at the center.



```
container: {  
  flexDirection: 'row',  
  justifyContent: 'space-between',  
  alignItems: 'center'  
}
```

3. flex-end — Children aligned at the end.



```
container: {  
  flexDirection: 'row',  
  justifyContent: 'space-between',  
  alignItems: 'flex-end'  
}
```

4. stretch (default) — Children stretched to fill up all space. This option doesn't work for children with fixed dimensions along the secondary axis.

CSS Modules

Another option for React Native styling is to create a.css file and import it into the component.

I made a basic styles.css file with the following styling:

```
h2 {  
  
  font-family: "Gill Sans", "Gill Sans MT", Calibri, "Trebuchet  
MS", sans-serif;  
  
  padding: 10px 5px;  
  
  border-radius: 10px;  
  
}
```

Then, in my House component file, import the file as shown below:

```
import React, { useState } from "react";  
  
import ReactDOM from "react-dom";  
  
import "../style.css"; //import here  
  
//keep everything else the same  
  
function House() {  
  
  const [color, setColor] = useState("red");  
  
  
  
  const style = {
```

```

        border: `1px solid ${color}`

    };

    return (

        <div>

            <h2 style={style}>This is a {color} house</h2>

        </div>

    );

}

```

React Props

You can use props — in React Native, props are what arguments are to a function and HTML attributes are to HTML.

Imagine we have a lot of Door components inside a house. How do we tell one Door apart from another? Why don't you give each Door a title prop?

To do so, simply add a title to the Door component as shown below.

```
<Door title="front" />
```

Kind of similar to [HTML](#) attributes, right? Then in the Door component, we can print out its prop:

```
function Door(props) {  
  return (  
    <div>  
      <h3>This is the {props.title} door</h3>  
    </div>  
  );  
}
```

The result? Our app would print the title out as expected.

Now, we can add lots of Door components inside the House and have a title to distinguish them.

```
function House() {  
  return (  
    <div>  
      <h2>This is a house</h2>  
      <Door title="front"/>  
      <Door title="back"/>  
      <Door title="kitchen"/>  
      <Door title="bedroom"/>  
    </div>  
  );  
}
```



```
}
```

React States

A state in React is an object that stores variables. You can only access these variables within its Component, unless you use some state management tools like [Redux](#).

Let's add some states to our House class component to make it more interesting:

```
class House extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      color: "white",  
      rooms: 4  
    };  
  }  
  render() {  
    return (  
      <div>  
        <h2>This is a {this.state.color} house with  
{this.state.rooms} rooms.</h2>  
      </div>  
    )  
  }  
}
```

useEffect()

The `useEffect` Hook is the next most helpful Hook you'll come upon. When a defined state changes, this hook executes a function.

Returning to our Home component, we add a variable named “door” to monitor the house’s other doors. We set it to 0 in its `useState` Hook.

Then, we add a button that, when clicked, increases the door's value by one. Finally, every time the door’s value is updated, we have a `useEffect` Hook that prints the number of doors in the home.

Here’s the code:

```
function House() {  
  
  const [color, setColor] = useState("white");  
  
  const [door, setDoor] = useState(0); //initialize door as 0  
  
  //add 1 to the current value of door on every button click  
  
  const addDoor = () => {  
  
    setDoor(door + 1);  
  
  };  
  
  useEffect(() => {  
  
    console.log(`Door count: ${door}`)  
  
  }, [door]);  
  
  return (  
  
    <div>  
  
      <h2>This is a {color} house</h2>  
  
      <button onClick={addDoor}>Add a Door</button>  
  
    </div>  
  
  );  
}
```

</div>

);

}